



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 02 — Gestión de procesos

Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



Comunicación con procesos

La idea en una frase

“

Un proceso hijo puede **leer de su stdin** y **escribir en su stdout**; con `communicate()` le pasas datos por un tubo y lees su respuesta por el otro.

Los procesos no comparten memoria, pero el sistema operativo les presta **pipes** (tuberías): el stdin, stdout y stderr de cada proceso pueden conectarse a Python. Así un proceso "pregunta" y el otro "responde".

Conectar los tubos

stdin=PIPE, stdout=PIPE

```
tubos.py Python
import subprocess
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)
```

- 1 `stdin=PIPE` : Python podrá **escribir** en el stdin del hijo
- 2 `stdout=PIPE` : Python podrá **leer** el stdout del hijo
- 3 `text=True` : los datos viajan como **texto**, no bytes

i El hijo ejecuta

`print(input().upper())` : lee una línea, la pasa a mayúsculas y la escribe en su stdout.

Enviar y recibir con communicate()

comunica.py

Python

```
import subprocess
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True)
salida, error = proceso.communicate(
    input="hola mundo")
print(f"Respondió: {salida.strip()}")
# → "HOLA MUNDO"
print(f"Error: {error}")
# → None (todo bien)
```

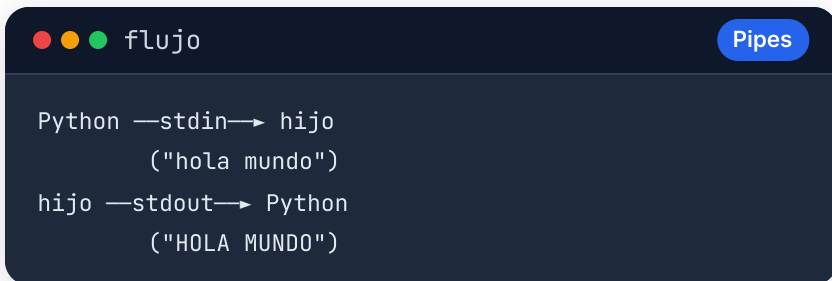
- 1 `communicate(input=...)` **escribe** en el stdin del hijo y cierra el tubo de entrada
- 2 **Espera** a que el hijo termine
- 3 Devuelve la tupla **(stdout, stderr)**



Después de `communicate()` el proceso ya terminó: **no puedes volver a comunicarte con él.**

El flujo de datos y los deadlocks

Flujo de datos



Cuidado con los deadlocks

Si el hijo escribe mucho en stdout y **nadie lo lee**, el pipe se llena y el hijo se bloquea esperando a que alguien lo vacíe.

✓ `communicate()` lee todo y te evita ese problema. Por eso con `run()` se usa `capture_output=True` y con `Popen`, `communicate()`.

¿Cuándo usar cada herramienta?

Situación	Herramienta
Necesitas el resultado de un comando y puedes esperar	<code>subprocess.run(..., capture_output=True)</code>
El proceso vive en segundo plano y no necesitas sus datos	<code>subprocess.Popen(...)</code> + <code>wait()</code>
Necesitas enviarle datos y/o leer su respuesta	<code>Popen(..., stdin=PIPE, stdout=PIPE)</code> + <code>communicate()</code>



Mini-chequeo: ¿qué significa `stdin=subprocess.PIPE` y `stdout=subprocess.PIPE`? ¿Qué devuelve `communicate(input="texto")`?

Que Python podrá **escribir** en el `stdin` del hijo y **leer** su `stdout` · `communicate()` devuelve la tupla **(`stdout`, `stderr`)**.

Compatibilidad Windows / Linux

La idea en una frase

“

Los ejemplos están escritos para **Windows**, pero los conceptos (subprocess.run, Popen, PID, communicate) son **idénticos en todos los sistemas**: solo cambia el comando.



`notepad.exe` no existe en Linux y `ls` no existe en Windows. Aprende a **sustituir el comando** y el código funciona igual.

La tabla de comandos equivalentes

Windows	Linux / macOS
<code>notepad.exe</code>	<code>gedit</code> , <code>nano</code> , <code>xed</code>
<code>calc.exe</code>	<code>gnome-calculator</code> , <code>bc</code>
<code>ping -n 5 8.8.8.8</code>	<code>ping -c 5 8.8.8.8</code>
<code>ipconfig</code>	<code>ip addr</code> , <code>ifconfig</code>
<code>where python</code>	<code>which python</code>
<code>cmd /c mkdir</code>	<code>mkdir</code> (es ejecutable real)
<code>start http:// ...</code>	<code>xdg-open http:// ...</code>
<code>dir</code>	<code>ls</code>

✓ Sustituye el comando correspondiente y el código funciona igual.

El truco del shell: mkdir en Windows

 En **Windows**, `mkdir` es un **comando interno de cmd**: no hay un `mkdir.exe`, hay que invocarlo con `cmd /c mkdir ...`

 En **Linux**, `mkdir` es un **ejecutable real** (`/usr/bin/mkdir`): se lanza directamente

 **shell=True con cuidado**: solo para comandos internos del shell (`start`, `dir`). Con texto del usuario, un atacante podría **inyectar comandos**. Pasa siempre la lista de argumentos.

```
shell.py Python
import subprocess
# Windows: cmd /c ejecuta comandos internos
r = subprocess.run(
    ["cmd", "/c", "mkdir", "prueba_psp"],
    capture_output=True, text=True)
print(f"Creada con código: {r.returncode}")
# En Linux: ["mkdir", "prueba_psp"] directo
```

Lo que nunca cambia

Concepto	Windows	Linux
<code>subprocess.run([...])</code>	✓	✓
<code>subprocess.Popen([...])</code>	✓	✓
<code>capture_output=True</code> , <code>text=True</code>	✓	✓
<code>timeout</code> , <code>returncode</code>	✓	✓
<code>communicate(input=...)</code>	✓	✓
<code>wait()</code> , <code>poll()</code> , <code>terminate()</code> , <code>kill()</code>	✓	✓
<code>proceso.pid</code>	✓	✓

- ✓ Aprende los conceptos con `notepad.exe` y `calc.exe` y, cuando toques Linux, solo cambia la lista de comandos.

Sé el código: dos aplicaciones a la vez

Traza cada paso antes de ejecutar

● ● ● doble.py

Python

```
import subprocess, time
print("🚀 Abriendo bloc de notas...")
notepad = subprocess.Popen(["notepad.exe"])
print(f" → PID: {notepad.pid}")
print("🚀 Abriendo calculadora...")
calc = subprocess.Popen(["calc.exe"])
print(f" → PID: {calc.pid}")
for i in range(5):
    print(f" Python haciendo cosas... ({i+1}/5)")
    time.sleep(1)
notepad.terminate()
calc.kill()
print("Hecho 🎉")
```

La traza, paso a paso

Tres procesos independientes conviviendo

1 `Popen(["notepad.exe"])` : el SO busca el ejecutable, lo carga en memoria, asigna **PID 12345** y Python recibe el control al instante

2 `Popen(["calc.exe"])` : lo mismo, **PID 12346**

3 Python hace 5 cosas mientras ambos están abiertos: **3 procesos independientes** (Python, notepad, calc)

4 `terminate()` cierra el bloc · `kill()` mata la calculadora

✓ **Python no se bloquea.** Mientras los dos programas están abiertos, Python sigue imprimiendo: cada uno con su PID, su memoria y su turno de CPU.

🧠 `run()` vs `Popen()`: el ring

i `run()`: "¡Soy el rey de la simplicidad! Lanzas, esperas y ¡zas! tienes el resultado."

i `Popen()`: "Mientras tú esperas, yo lanzo y sigo. ¿3 programas a la vez? Los lanzo, me voy a café y luego los mato."

⚠ **Moraleja:** `run()` para comandos rápidos que necesitan respuesta. `Popen()` para procesos que deben vivir en segundo plano.

Mini-chequeo: práctica



Las tres del ring:

1. En el "Sé el código", ¿cuántos procesos hay vivos mientras Python imprime sus 5 mensajes?
2. ¿Cuándo usarías `run()` y cuándo `Popen()`?
3. ¿Qué devuelve `os.getppid()` ?



Respuestas:

- **3:** Python, el bloc de notas y la calculadora
- `run()` para respuestas inmediatas; `Popen()` para segundo plano
- El **PID del proceso padre** (el que lanzó tu programa)

Sé el proceso: eres el PID 12345

Nace, vive y muere un proceso



Eres un proceso de Python recién lanzado con

`subprocess.Popen(["python", "calcula.py"])`. Acaban de asignarte el PID 12345. ¿Qué te pasa?

- 1 El SO crea tu proceso: PID **12345**, burbuja de memoria reservada
- 2 **NUEVO** → **LISTO**: esperas turno en la cola del planificador
- 3 La CPU te toca: **EJECUCIÓN**. Ejecutas `calcula.py`
- 4 Llamas a `input()`: te bloqueas (**BLOQUEADO**) esperando al usuario
- 5 El usuario escribe y pulsa ENTER: la E/S termina y vuelves a **LISTO**
- 6 La CPU te toca otra vez (**EJECUCIÓN**) y terminas: **TERMINADO**
- 7 Tu padre recoge tu código de retorno con `wait()/poll()`: **0**



Fireside chat: Proceso vs Hilo

Quién es más ligero

i Proceso: — Yo soy la unidad completa: memoria propia, recursos, PID. Si me cuelgo, tú ni te enteras.

✓ Hilo: — ¿Y para qué tanta burbuja? Yo vivo **dentro** de un proceso y comparto su memoria. Soy mucho más barato de crear.

i Proceso: — Mis fallos no tumban a nadie. Tú, si se te va un puntero, te llevas por delante al proceso entero.

✓ Hilo: — Cierto, pero yo comparto variables directamente. Tú necesitas pipes y `communicate()` para hablar con tu vecino.

i Proceso: — Y mi paralelismo con multiprocessing es real: los hilos comparten el **GIL**.

⚠ Moraleja: el proceso **aisla y paraleliza de verdad**; el hilo es **ligero y comparte memoria**. Los hilos los veremos más adelante en el temario.



¿Quién soy?



Endevina qué concepto de la unidad soy:

1. Soy el identificador único que el SO asigna a cada proceso
2. Soy el estado en el que el proceso espera un recurso (I/O, socket, sleep)
3. Soy el mecanismo por el que una sola CPU parece ejecutar varios procesos a la vez
4. Soy el módulo de Python que reparte tareas en procesos paralelos reales
5. Soy el tubo que conecta la salida de un proceso con la entrada de otro
6. Soy el proceso que ya terminó pero que su padre no ha recogido todavía

 **Respuestas:** 1) **PID** · 2) **BLOQUEADO** · 3) El **planificador** con la concurrencia (time slices) · 4) **multiprocessing** · 5) El **pipe** · 6) El **zombie**



CONRAD vs el mundo: "maté el proceso y no volvió"

⚠️ *"He lanzado mi programa con `subprocess.run` y no me devuelve el control."* ¡Pues claro! `run()` espera a que el comando termine: si lanzas `notepad.exe`, no se cierra nunca. Si querías lanzar y seguir, eso es `Popen`, no `run()`.

⚠️ *"Puse `shell=True` para abrir el navegador y ahora un comando borra mis archivos."* `shell=True` delega en el intérprete: con datos del usuario es un **agujero de seguridad**. Lista de argumentos, nunca strings concatenados. O `cmd /c`, que para eso está.

⚠️ *"¿Será que mi proceso se ha convertido en zombie?"* Míralo con `proceso.poll()`: si devuelve el código de retorno, ya terminó y solo falta recogerlo con `wait()` o `poll()`. **Zombie sin recoger = entrada ocupada** en la tabla de procesos. A diagnosticar.



Atrévete a pensar

- 1 ¿Por qué un proceso no puede compartir memoria directamente con otro?
- 2 ¿Qué pasa si un hijo muere y su padre nunca llama a `wait()` ni `poll()`?
- 3 ¿Cuándo usarías `multiprocessing` en lugar de `subprocess`?
- 4 ¿Por qué `shell=True` es peligroso con datos del usuario?
- 5 ¿Qué ventaja tiene `Popen` sobre `run()` para lanzar 3 aplicaciones a la vez?

💡 Soluciones

- 1) Burbuja aislada: se comunican con mecanismos externos (pipes, sockets, archivos)
- 2) Queda **zombie**: su entrada sigue en la tabla de procesos
- 3) `multiprocessing` para **paralelismo real**; `subprocess` para **lanzar programas**
- 4) Un dato con `;`, `,` `&` o `|` ejecuta comandos extra (inyección)
- 5) `Popen` devuelve el control al instante



Preguntas de entrevista de trabajo



Preguntas reales de programador Python junior:

1. ¿Qué es un proceso y en qué se diferencia de un programa?
2. Explica el ciclo de vida de un proceso
3. ¿Qué diferencia hay entre computación paralela y distribuida?
4. ¿Cómo lanzarías un programa externo desde Python y leerías su salida?
5. ¿Qué diferencia hay entre `run()` y `Popen()`? ¿Cuándo usarías cada uno?
6. ¿Cómo comunicarías dos procesos entre sí?



Las "preguntas reina" son la 4 y la 5: `run([...], capture_output=True, text=True)` → `resultado.stdout` · y la moraleja: `run()` para respuestas inmediatas, `Popen()` para segundo plano.

Resumen — Bloque 02 completado

Lo que hemos aprendido

Comunicación

stdin/stdout con PIPE · `communicate(input=...)` ·
deadlocks

Windows / Linux

Misma API, otro comando · `cmd /c` y `shell=True` con
cuidado

Práctica

Traza de procesos conviviendo · `run()` vs `Popen()` ·
zombie

Cierre

Proceso vs hilo · ¿quién soy? · preguntas de entrevista

✓ **Ya sabes lanzar, vigilar, alimentar y matar procesos desde Python. Ahora: boletines de práctica.** 

